

Database, Audit, M-Pesa & Portal Implementation Guide

Supporting specification for the clean Next.js + Supabase rebuild

Decision: Build on the new Supabase account. Apply the accompanying SQL only to a disposable development project after review. The legacy database remains a migration source, never the new foundation.

Companion migration

0001_hakika_platform_foundation.sql

Prepared for

Codex implementation

Date

11 August 2026

1. What Codex should build first

The first backend milestone is not every module table. It is a secure platform kernel that makes every later table inherit the same identity, tenancy, permission, audit and payment rules.

- Create PostgreSQL schemas: platform, iam, billing, audit, communications, integrations, real_estate and private.
- Create canonical organizations, companies, memberships, application subscriptions, pages, roles and permissions.
- Create an immutable business audit stream plus separate low-risk UI telemetry.
- Create Real Estate reference, monthly billing, payment allocation, split, M-Pesa, portal and messaging foundations.
- Enable RLS and prove isolation with two organizations before building dashboards.
- Generate database types only after migrations pass in the new development project.

Do not push this migration to a production Supabase project in the first run. Codex must produce a dry-run report, SQL review, pgTAP results and rollback notes first.

2. Canonical tenancy and access model

auth.users → iam.profiles → organization membership → company membership → application subscription → application/page role → permission

Concept	Rule
Organization	Hard tenant boundary. Organization 0 is the single platform-owner organization.
Company	Operational subdivision belonging to one organization.
Application	Stable uppercase key such as REAL_ESTATE, HR or TOUGHFORCE.
Subscription	Determines whether an organization is entitled to an application.
Role	A named permission bundle; may be platform, organization or application scoped.
Page permission	Controls read/create/update/delete/approve/export/manage/portal-preview actions.
Read-only user	Receives roles containing read permissions only; UI hiding is secondary to RLS/server enforcement.

3. Platform audit: record the story, not meaningless noise

Every meaningful business or security action creates one immutable audit.events row. Platform Admin displays it as a human-readable timeline, for example: “John Doe created employee Jane Wanjiku in Hakika Group / Default Company.” The record still retains machine fields for filtering and investigation.

Audit field	Purpose
Actor snapshot	User ID plus name/email at event time, so history survives profile edits.
Tenant context	Organization and company IDs and resolved names in the feed.
Application/page/action	Where the action happened and the stable action key.
Entity	Type, UUID and readable label of the affected record.
Outcome	Success, failure or denied.
Before/after	Redacted JSON for approved mutable fields; changed_fields for compact display.
Request/correlation	Joins UI request, database write, background job and external callback.
Summary	Ready-to-display sentence, generated server-side from trusted values.

Never audit secrets: passwords, reset/invitation tokens, access tokens, M-Pesa credentials, raw identity documents, full payment credentials or private message bodies. Audit that a message was sent, its recipient

and delivery result - not its sensitive content.

3.1 Business audit vs UI telemetry

Stream	Examples	Retention/use
audit.events	Created employee; issued invoice; changed role; opened portal preview; approved payout; login denied	Immutable compliance/security record; visible in authorized admin timelines
audit.ui_events	Opened filter; changed tab; clicked export before request	Controlled product analytics; sampled/retained separately; never used for authorization

A blanket “record every click forever” rule would flood the feed and collect unnecessary personal behavior. Codex should use an allowlisted event catalogue and classify events as business, security or analytics.

4. Permission and invitation behavior

- Admin invites a user by email and assigns organization, one or more companies, applications and roles.
- Invitation tokens are opaque, short-lived and stored only as hashes.
- Acceptance is idempotent: it links or creates the auth identity exactly once.
- Page visibility comes from the page registry and read permission, but every server mutation and RLS policy checks the matching action permission.
- Read-only is a permission set, not a fragile boolean bypass. A read-only role has no create/update/delete/approve/manage permissions.
- Changing or revoking access invalidates the canonical access snapshot and records an audit event.

Example permission keys

```
real_estate.units.read real_estate.units.create real_estate.units.update real_estate.billing.approve
real_estate.payments.allocate real_estate.tenant_portal.preview admin.members.invite
platform.audit.read
```

5. Portal architecture

Portal	Primary visibility
Tenant portal	Own profile, active/past leases, invoices, receipts, balance, maintenance, documents and messages.
Landlord portal	Assigned properties/units, occupancy, approved expenses, statements, splits and payouts.
Caretaker portal	Assigned properties, units, tenants as permitted, inspections, maintenance, notices and messages.

Portal users use normal Supabase identities and explicit `real_estate.portal_grants`. A grant maps the user to a tenant, landlord or caretaker entity inside one organization. Portal RLS must validate that mapping; it must not infer portal access from matching email or phone values.

5.1 Admin “open portal in new tab”

Do not impersonate the tenant invisibly. Use a short-lived, single-purpose support-preview session. The admin keeps their own identity; the portal receives a signed/hashed preview token scoped to organization, portal type and entity. The header clearly states “Admin preview - read only” unless an approved support workflow explicitly allows an action. Creation, opening, use and expiry are audited.

5.2 Tenant message history

- Conversation participants include users, tenants, landlords, caretakers or teams.
- Messages are stored once; delivery rows track in-app, SMS, email or WhatsApp states.
- The tenant detail page queries all authorized conversations containing that tenant participant.
- The portal shows only conversations permitted by its grant and organization scope.
- Audit logs metadata and delivery outcome, while message content remains in the communications domain.

6. Monthly auto-billing

The required invariant is one consolidated recurring invoice per active lease/tenant billing relationship per calendar month. Charges such as rent, service charge, water, penalties and prior adjustments become invoice items on that single invoice.

```
UNIQUE (organization_id, lease_id, billing_month)
```

Object	Responsibility
Billing schedule	Timezone, issue day, issue time, due day and approval mode.
Reminder rule	Offset days, send time, channel and template. Negative offset means before due date.
Invoice run	One idempotent job per company/schedule/month; preview, approval and failure state.
Invoice	One lease/month identity with totals and balance.
Invoice items	Rent and all other line-level charges.

Recommended job flow

- Scheduler enqueues a deterministic month/company job.
- Worker obtains an advisory/idempotency lock.
- Build preview and exceptions without issuing invoices.
- Authorized user approves, unless the organization has deliberately enabled automatic issue.
- Upsert invoice identity and items in a transaction.
- Issue documents and queue reminders.

- Record one audit event for the run and events for exceptions; never double-create on retry.

Use Africa/Nairobi as the default timezone, but store organization timezone explicitly. Limit issue/due day to 1-28 initially to avoid month-end ambiguity.

7. Safaricom Business One Account and Daraja

Safaricom describes Business One Account as using one Lipa na M-PESA shortcode to collect and disburse funds, and states that PayBill and B2C services are prerequisites. Safaricom also exposes Daraja APIs for C2B, B2C, B2B, transaction query, account balance, M-Pesa Express and reversals. This supports a shared rail, but Hakika still needs its own sub-ledger and payout controls.

7.1 Recommended architecture

```
Customer payment → Business One Account shortcode → C2B/STK callback → immutable M-Pesa transaction → reference resolution → organization/application/entity sub-ledger → invoice allocation → versioned split allocation → approved payout job → B2C/B2B → status/reconciliation
```

Important inference: the reviewed official pages describe collection/disbursement APIs, not a complete multi-tenant property split-ledger service. Therefore, Hakika should calculate and approve landlord/platform/tax splits internally, then send idempotent payout instructions through the appropriate Safaricom API. Confirm final product capabilities, tariffs, limits and approval workflow with the assigned Safaricom integration team before production.

7.2 Reference strategy

- Generate a stable customer reference that resolves to organization, company, application and entity without exposing raw UUIDs.
- Validate the maximum account-reference format and length with Safaricom before locking the format.
- Never trust payer phone alone for allocation. Use exact reference, expected amount/tenant context and a manual exception queue.
- Unknown or ambiguous transactions remain unallocated; they never leak into a default organization.
- One provider receipt is unique and immutable. Duplicate callbacks return success after detecting the prior callback/transaction.
- Store raw callbacks in a protected integration table for reconciliation, but expose redacted transaction views to application users.

7.3 Split settlement controls

- Split rules are versioned with effective dates and approval metadata.
- Every split allocation points to the exact version used.
- Allocated amounts must equal the approved distributable amount before payout.
- Payout jobs use stable idempotency keys, bounded retries and transaction-status queries for unknown results.
- Never retry an unknown payout blindly; query status first to prevent duplicate disbursement.
- Reversals and refunds create compensating ledger entries rather than deleting history.
- Require maker-checker approval above configurable thresholds.

8. Core database domains

Schema	Key tables
platform	organizations, companies, applications, provisioning jobs
iam	profiles, memberships, pages, permissions, roles, invitations
billing	plans and application subscriptions
audit	events, UI events and platform event feed
real_estate	properties, units, tenants, landlords, caretakers, leases, billing, payments, splits, portals
integrations	M-Pesa accounts, references, callbacks, transactions and payout jobs
communications	conversations, participants, messages and deliveries
private	fixed-search-path authorization and audit helper functions

9. RLS requirements

- All tenant-owned tables enable RLS before application use.
- Self-read membership policies allow the authenticated user to resolve their own context.
- Every company-owned query validates both organization and company membership.
- Application subscription and action permission are separate checks.
- INSERT/UPDATE use WITH CHECK so a permitted user cannot move a row into another tenant.
- Portal policies validate portal_grants and entity ownership.
- Platform operators do not receive a magical profile-role bypass; they receive explicit platform-owner membership and platform permissions.
- Service role is restricted to reviewed server workflows; user-facing reads still use scoped views/RPCs.

10. Required tests before any push

- Create Organization A and B, companies, users and subscriptions; prove every cross-tenant read/write/reference fails.
- Prove read-only roles cannot mutate through UI, server action, RPC or direct Supabase client.
- Prove invitation acceptance is idempotent and expired/revoked tokens fail.
- Prove repeated auth events create one access bootstrap, not a loop.
- Prove repeated monthly billing jobs create one invoice per lease/month.
- Prove duplicate M-Pesa callbacks create one provider transaction.
- Prove unallocated payments do not change an invoice balance.
- Prove payment allocations cannot exceed the payment or invoice balance.
- Prove split allocations balance and retain their rule version.
- Prove unknown payout results are status-queried before retry.
- Prove admin portal preview expires, is clearly labeled and is audited.
- Prove customer audit readers cannot see another organization; platform audit readers can filter globally only with explicit permission.

11. Codex start instruction

You are implementing the clean Hakika Business OS foundation in a NEW repository and NEW Supabase project. Read these files first:

1. Hakika_Business_OS_Complete_Rebuild_Specification.docx
2. Hakika_Database_Audit_Mpesa_and_Portal_Implementation_Guide.pdf
3. 0001_hakika_platform_foundation.sql

Start Phase 1 only. Do not connect to or mutate the legacy Supabase project.

Tasks:

- Create the frontend/, backend/, packages/, docs/ and tests/ root structure documented in the specification.
- Initialize Next.js App Router, strict TypeScript, pnpm workspace and shared UI/config packages.
- Configure the new Supabase project through environment variables; commit only .env.example.
- Review the SQL line by line. Correct dependency order, foreign keys, indexes, grants, schema exposure and PostgreSQL/Supabase compatibility. Split it into ordered migrations by domain instead of pushing one giant migration.
- Add complete rollback notes and pgTAP tests.
- Seed stable application/page/permission keys and system roles.
- Implement server-first auth with one browser client, one listener and one access snapshot.
- Implement Home plus independent /products, /pricing, /security, /about, /contact and /support pages.
- Implement login, organization/company selection and /apps launcher.
- Implement separate Platform Admin and Customer Admin route/permission boundaries.
- Add audit event service and Platform Admin audit feed. Do not log secrets or every raw form value.
- Add invitation flow with app/page roles and read-only roles.
- Add tenant, landlord and caretaker portal grant foundations and read-only admin preview sessions.
- Design M-Pesa integration interfaces and mocks only in Phase 1; never store secrets in database rows or call production Daraja.

Before applying anything, return:

- A. files created;
- B. migration order and what each migration does;
- C. security/RLS review;
- D. test plan and dry-run output;
- E. unresolved Safaricom onboarding questions;
- F. explicit statement that no production database was changed.

Stop for approval before supabase db push.

12. Safaricom questions to resolve before production

- Confirm the exact Business One Account shortcode(s), linked PayBill and B2C accounts, and production Daraja app ownership.
- Confirm supported C2B callback registration, validation behavior, IP/TLS requirements and callback retry behavior.
- Confirm account-reference length/character limits and whether store/account routing features apply.
- Confirm B2C/B2B transaction limits, fees, settlement timing, initiator/security credential requirements and maker-checker expectations.
- Confirm transaction-status, balance and reversal permissions for the account.
- Confirm whether funds belonging to independent customer organizations may legally/contractually be collected and disbursed through this one account.
- Complete financial, tax, trust/escrow and data-protection review before holding or splitting third-party funds.

13. Official Safaricom references reviewed

Daraja Developer Portal

<https://developer.safaricom.co.ke/>

Customer to Business API

<https://developer.safaricom.co.ke/apis/CustomerToBusiness>

Business to Customer API

<https://developer.safaricom.co.ke/apis/BusinessToCustomer>

Account Balance API

<https://developer.safaricom.co.ke/apis/AccountBalance>

Reversal API

<https://developer.safaricom.co.ke/apis/Reversal>

Business One Account

<https://www.safaricom.co.ke/main-mpesa/for-your-business/business-one-account>

M-Pesa business channels and APIs

<https://www.safaricom.co.ke/main-mpesa/for-your-business/business-channels>

Lipa na M-Pesa / Business One Account summary

<https://www.safaricom.co.ke/main-mpesa/m-pesa-services/lipa-na-mpesa>

Research note: official pages confirm available product/API categories. Exact commercial configuration, limits and production credentials remain account-specific and must be confirmed with Safaricom.